

CANDIDATE NOTES · START FROM ZERO

The Road to DevOps

Computer basics · Linux (Ubuntu) commands · Software Development Life Cycle
· Waterfall · Agile · Why DevOps

Simple notes for someone who has never studied computers. Read one chapter at a time.
Every chapter ends with a short summary and a few questions to check yourself.

Contents

How to read these notes	4
The journey	4
How the notes are written	4
Computer basics	5
What is a computer?	5
What is inside a computer?	5
Hardware and software	6
The operating system (OS)	6
Applications and programs	7
Files and folders	7
Networks and the internet	7
Linux and Ubuntu commands	9
What is Linux? What is Ubuntu?	9
How to get Ubuntu on your laptop	9
The terminal	9
1. Moving around	10
2. Files and folders	11
3. Reading files	11
4. Writing and editing	11
5. Searching	12
6. Users, sudo, and permissions	12
7. Installing software (apt)	13
8. Running programs and services	13
9. System information	14
10. Network commands	14
11. Compressing files	14
12. Getting help and handy shortcuts	15
Practice session	15
The Software Development Life Cycle (SDLC)	17
What is the SDLC?	17

The seven phases	17
People in a software team	18
Important words	18
SDLC models	18
The Waterfall model	20
How Waterfall works	20
Advantages	20
Disadvantages	20
When to use Waterfall	21
The Agile model	22
Why Agile was created	22
How Agile works	22
The four Agile values	22
Scrum: the most popular Agile method	22
Kanban: another Agile method	23
Waterfall vs Agile	24
The problem Agile did not solve	24
Why DevOps?	26
What is DevOps?	26
Waterfall → Agile → DevOps	26
The problems DevOps solves	27
The DevOps life cycle	27
Key DevOps practices	28
Benefits of DevOps	28
Who uses DevOps?	29
What a DevOps engineer does	29
Your learning path from here	29
One-page revision sheet	30
Answers to “Check yourself”	31
Glossary	32

WELCOME

How to read these notes

You do not need any technical background. If you can use a phone and follow a recipe, you can follow these notes.

The journey

These notes walk you step by step towards DevOps. Each chapter builds on the one before it:



How the notes are written

- New technical words are in **bold**. Things you type into a computer look like `this`.
- **Think of it like this** boxes explain ideas with everyday examples.
- **Try it yourself** boxes are small exercises. Doing them is the fastest way to learn.
- **In short** boxes at the end of each chapter are your revision notes.
- Answers to the questions are at the end of the notes.

Key idea

Do not try to memorise everything. Aim to **understand** the idea and be able to explain it to a friend in your own words.

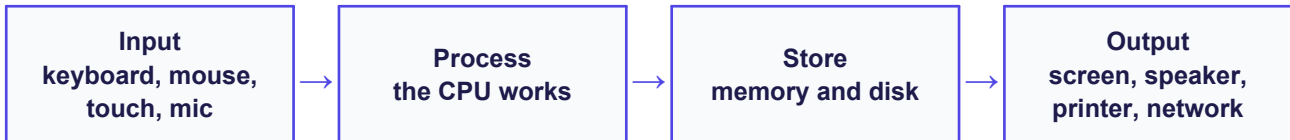
CHAPTER 1

Computer basics

What a computer is, what is inside it, and the words every IT person uses.

What is a computer?

A **computer** is a machine that follows instructions. It takes something in, works on it, remembers it if needed, and gives a result back.



Example: you type 2 + 3 on a calculator app (input), the computer adds it (process), and shows 5 (output).

What is inside a computer?

Think of it like this

Imagine a computer is a **small office**:

- The **CPU** is the worker who does all the work.
- The **RAM** is the worker's desk — fast to use, but cleared every evening.
- The **hard disk / SSD** is the cupboard — slower, but files stay there for years.
- The **operating system** is the office manager who organises everything.

Part	Simple meaning	Common sizes
CPU (processor)	The “brain”. Does calculations and runs programs.	Intel Core i5, AMD Ryzen, Apple M-series
RAM (memory)	Short-term memory. Holds what you are working on right now. Empties when power is off.	8 GB, 16 GB, 32 GB
Storage (HDD / SSD)	Long-term memory. Keeps files, photos, and apps even when power is off. SSD is faster than HDD.	256 GB, 512 GB, 1 TB
Motherboard	The main board that connects all parts together.	—
Network card	Connects the computer to Wi-Fi or a cable network.	Wi-Fi, Ethernet

Units of size

The smallest unit is a **bit** (a 0 or a 1). **8 bits make 1 byte**, which is enough to store one letter, such as A.

Unit	Equal to	Example
1 Byte	8 bits	One letter or digit
1 KB (kilobyte)	1,024 bytes	A short text message
1 MB (megabyte)	1,024 KB	A photo from a phone (2–5 MB)
1 GB (gigabyte)	1,024 MB	A full-length movie (1–4 GB)
1 TB (terabyte)	1,024 GB	Hundreds of movies

Key idea

Computers count in powers of 2, so each unit is **1,024** times the one before it (2 to the power 10). You may sometimes see 1,000 used instead, for example on hard-disk boxes — that is why a “1 TB” disk shows a little less space in your computer.

Hardware and software

	Hardware	Software
Meaning	The physical parts you can touch	Instructions that tell hardware what to do
Examples	Screen, keyboard, CPU, RAM, mouse	Windows, Chrome, WhatsApp, Excel
Analogy	The body	The thoughts and skills

The operating system (OS)

The **operating system** is the main software that runs the computer. It starts when you switch on, manages the hardware, and lets you open other apps.

Operating system	Where you see it
Windows	Most office laptops and desktops
macOS	Apple MacBooks and iMacs
Android / iOS	Mobile phones
Linux (Ubuntu, Red Hat, Debian...)	Most servers on the internet, cloud computers, and DevOps tools

Key idea

Most of the internet runs on **Linux**. That is why every DevOps engineer must be comfortable with Linux. Chapter 2 teaches you the basics.

Applications and programs

- An **application (app)** is software that does one job for you: a browser, a music player, a calculator.
- A **program** is a set of instructions written by a programmer. Every app is made of one or more programs.
- **Code** is the written text of a program, in a **programming language** such as Python, Java, or JavaScript.

Think of it like this

Code is like a recipe. The programmer writes the recipe once. The computer is the cook that follows it exactly — every time, without guessing.

Files and folders

A **file** is a saved piece of information, like a document, photo, or song. A **folder** (in Linux called a **directory**) holds files and other folders. The written location of a file is called its **path**.

```
Windows:  C:\Users\Ravi\Documents\resume.pdf
Linux:    /home/ravi/documents/resume.pdf
```

The letters after the dot are the **extension**. They tell you the file type: `.pdf`, `.jpg`, `.txt`, `.mp4`, `.sh` (a Linux script).

Networks and the internet

Word	Simple meaning
Network	Two or more computers connected so they can share information
Internet	A worldwide network of networks
Server	A computer that provides a service to other computers, such as a website or email
Client	The computer or phone that asks the server for something
IP address	A number address for a device on a network, for example <code>192.168.1.10</code>
Domain name	An easy name for a website, for example <code>google.com</code>
DNS	The internet's phone book. It converts a domain name into an IP address
Cloud	Using someone else's computers (like Amazon's) over the internet, and paying for what you use

Think of it like this

A **server** is like a restaurant kitchen and a **client** is the customer. The customer orders, the kitchen prepares, and the waiter (the network) carries the order and food.

In short

A computer takes input, processes it, stores it, and gives output.

CPU = brain, RAM = desk (temporary), storage = cupboard (permanent).

Hardware is physical; software is instructions. The operating system manages everything.

Servers on the internet mostly run Linux.

Check yourself

- 1 What happens to the data in RAM when you switch off the computer?
- 2 Give two examples of hardware and two of software.
- 3 Which operating system do most servers use?
- 4 What does DNS do?

Linux and Ubuntu commands

How to control a Linux computer by typing commands. This is the most important practical skill before DevOps.

What is Linux? What is Ubuntu?

Linux is a free operating system, like Windows, but used mostly on servers. Different companies package Linux in different ways; each package is called a **distribution** (or “distro”).

Ubuntu is the most popular, beginner-friendly Linux distribution. It is free and widely used in the cloud.

Why Linux is used for servers	
Free	No licence cost, even for thousands of servers
Stable	Servers can run for months without restarting
Secure	Strong user and permission system
Light	Can run without a screen or mouse, which saves resources
Automation friendly	Everything can be done by typed commands, so it can be scripted

How to get Ubuntu on your laptop

- **Windows (easiest):** open **PowerShell as Administrator** and type `wsl --install`. Restart, then open **Ubuntu** from the Start menu. This is called WSL (Windows Subsystem for Linux).
- **Cloud:** start a free-tier Ubuntu server on AWS and connect to it from the browser.
- **Virtual machine:** install VirtualBox and run Ubuntu inside it.

The terminal

The **terminal** is a window where you type commands. The text that waits for your command is called the **prompt**:

```
ravi@my-laptop:~$
```

Part	Meaning
ravi	Your username
my-laptop	The computer's name (hostname)

Part	Meaning
~	Where you are now. ~ means your home folder
\$	You are a normal user. A # means you are the all-powerful admin user (root)

Think of it like this

Clicking icons is like pointing at pictures on a menu. Typing commands is like **speaking your order** — faster once you know the words, and you can write it down and repeat it any time.

How a command is written

```
command  -options  target
ls       -l        /home
```

The **command** is the action, **options** (also called flags) change how it behaves, and the **target** is what it acts on.

1. Moving around

Linux folders form a tree that starts at /, called the **root** folder.

```
/
├── home/      the top of everything
│   └── ravi/   personal folders of users (like "My Documents")
├── etc/       settings (configuration) files
├── var/       changing data, such as logs in /var/log
├── bin/       basic commands
└── tmp/       temporary files
```

Command	What it does	Example
pwd	Shows where you are (print working directory)	pwd
ls	Lists files and folders	ls
ls -l	Lists with details: size, date, permissions	ls -l
ls -a	Shows hidden files too (names starting with .)	ls -a
cd	Goes into a folder (change directory)	cd documents
cd ..	Goes up one folder	cd ..
cd ~	Goes to your home folder	cd ~
cd /	Goes to the root folder	cd /var/log

2. Files and folders

Command	What it does	Example
mkdir	Makes a new folder	mkdir projects
mkdir -p	Makes folders inside folders in one go	mkdir -p app/logs/2026
touch	Creates an empty file	touch notes.txt
cp	Copies a file	cp notes.txt notes-backup.txt
cp -r	Copies a whole folder	cp -r projects projects-copy
mv	Moves or renames a file	mv notes.txt todo.txt
rm	Deletes a file	rm todo.txt
rm -r	Deletes a folder and everything inside	rm -r projects-copy
rmdir	Deletes an empty folder	rmdir old

Be careful

Linux has **no recycle bin**. `rm` deletes for ever. Always check with `pwd` and `ls` before deleting, and never run `rm -rf /`.

3. Reading files

Command	What it does	Example
cat	Shows the whole file	cat notes.txt
less	Shows a long file page by page (q to quit)	less /var/log/syslog
head	Shows the first 10 lines	head -n 5 notes.txt
tail	Shows the last 10 lines	tail notes.txt
tail -f	Keeps showing new lines as they arrive (great for logs)	tail -f app.log
wc -l	Counts the lines in a file	wc -l notes.txt

4. Writing and editing

Command	What it does	Example
echo	Prints text on the screen	echo "Hello"
>	Writes output into a file (replaces it)	echo "Line 1" > a.txt

Command	What it does	Example
>>	Adds output to the end of a file	echo "Line 2" >> a.txt
nano	A simple text editor inside the terminal	nano a.txt

In **nano**: type normally, press **Ctrl + O** then **Enter** to save, and **Ctrl + X** to exit.

5. Searching

Command	What it does	Example
grep	Finds lines that contain a word	grep "error" app.log
grep -i	Same, ignoring upper/lower case	grep -i "error" app.log
find	Finds files by name	find . -name "*.txt"
(pipe)	Sends the output of one command into another	ls -l grep txt

Think of it like this

The **pipe** | is like a factory conveyor belt: the first machine's output becomes the next machine's input.

6. Users, sudo, and permissions

Linux has normal users and one super user called **root** who can do anything. To run a single command as an administrator, put **sudo** ("super user do") in front of it. It will ask for your password.

Command	What it does	Example
whoami	Shows your username	whoami
sudo	Runs a command as administrator	sudo apt update
sudo adduser	Creates a new user	sudo adduser priya
passwd	Changes your password	passwd
chmod	Changes who can read, write, or run a file	chmod +x script.sh
chown	Changes the owner of a file	sudo chown priya notes.txt

Reading permissions

Run `ls -l` and you will see something like this at the start of each line:

```
-rwxr-xr-- 1 ravi ravi 220 Oct 1 10:00 script.sh
```

Letters	Who	Meaning
rwx	Owner (ravi)	can read, write, and execute (run)
r-x	Group	can read and run, but not change
r--	Everyone else	can only read

r = read, **w** = write (change), **x** = execute (run). A dash - means “not allowed.”

7. Installing software (apt)

Ubuntu installs software with **apt**, the package manager. Think of it as the Play Store for the terminal.

Command	What it does	Example
<code>sudo apt update</code>	Refreshes the list of available software	<code>sudo apt update</code>
<code>sudo apt upgrade</code>	Updates installed software	<code>sudo apt upgrade -y</code>
<code>sudo apt install</code>	Installs a program	<code>sudo apt install tree</code>
<code>sudo apt remove</code>	Removes a program	<code>sudo apt remove tree</code>

8. Running programs and services

A running program is called a **process**. A program that runs in the background all the time (like a web server) is called a **service**.

Command	What it does	Example
<code>ps aux</code>	Lists all running processes	<code>ps aux grep nginx</code>
<code>top</code>	Live view of CPU and memory use (q to quit)	<code>top</code>
<code>kill</code>	Stops a process by its number (PID)	<code>kill 1234</code>
<code>systemctl status</code>	Shows if a service is running	<code>systemctl status nginx</code>

Command	What it does	Example
<code>systemctl start / stop</code>	Starts or stops a service	<code>sudo systemctl restart nginx</code>
<code>systemctl enable</code>	Starts a service automatically at boot	<code>sudo systemctl enable nginx</code>

9. System information

Command	What it does	Example
<code>uname -a</code>	Shows Linux version details	<code>uname -a</code>
<code>lsb_release -a</code>	Shows the Ubuntu version	<code>lsb_release -a</code>
<code>df -h</code>	Shows free disk space	<code>df -h</code>
<code>du -sh</code>	Shows the size of a folder	<code>du -sh /var/log</code>
<code>free -h</code>	Shows free memory (RAM)	<code>free -h</code>
<code>uptime</code>	Shows how long the computer has been on	<code>uptime</code>

The `-h` option means “human readable”: sizes are shown in MB and GB instead of long numbers.

10. Network commands

Command	What it does	Example
<code>ip a</code>	Shows your IP addresses	<code>ip a</code>
<code>ping</code>	Checks if another computer is reachable	<code>ping -c 4 google.com</code>
<code>curl</code>	Fetches a web page or calls a web address	<code>curl https://example.com</code>
<code>wget</code>	Downloads a file	<code>wget https://example.com/file.zip</code>
<code>ssh</code>	Logs in to another Linux computer securely	<code>ssh ubuntu@13.233.10.5</code>

11. Compressing files

Command	What it does	Example
<code>tar -czf</code>	Packs a folder into one compressed file	<code>tar -czf backup.tar.gz projects</code>
<code>tar -xzf</code>	Unpacks a compressed file	<code>tar -xzf backup.tar.gz</code>
<code>zip / unzip</code>	Works with .zip files	<code>unzip file.zip</code>

12. Getting help and handy shortcuts

Command	What it does	Example
man	Opens the manual for a command (q to quit)	man ls
--help	Shows a short help text	ls --help
history	Shows commands you typed before	history
clear	Clears the screen	clear

Shortcut	What it does
Tab	Completes file and command names for you — use it all the time
↑ / ↓ arrows	Shows previous commands
Ctrl + C	Stops the command that is running
Ctrl + L	Clears the screen

Practice session

Try it yourself

Type these one by one and watch what happens:

```
pwd                                # where am I?
mkdir -p ~/practice/logs          # make folders
cd ~/practice                     # go inside
echo "Day 1 of Linux" > notes.txt  # create a file with text
echo "I learned ls and cd" >> notes.txt
cat notes.txt                     # read it
cp notes.txt logs/notes-copy.txt  # copy it
ls -l logs                        # check the copy
grep "ls" notes.txt               # search inside the file
sudo apt update                   # refresh software list
sudo apt install tree -y          # install a small tool
tree ~/practice                   # see your folders as a tree
df -h                             # check disk space
free -h                           # check memory
cd ~ && rm -r ~/practice           # clean up
```

In short

Ubuntu is a beginner-friendly Linux. The terminal lets you control it by typing.

Move: `pwd`, `ls`, `cd`. Files: `mkdir`, `touch`, `cp`, `mv`, `rm`. Read: `cat`, `less`, `tail`.

Admin power: `sudo`. Install software: `sudo apt install`. Services: `systemctl`.

Check the system: `df -h`, `free -h`, `top`. Network: `ip a`, `ping`, `curl`, `ssh`.

Check yourself

- 1 Which command shows the folder you are in?
- 2 How do you create a folder called `website` and go into it?
- 3 What is the difference between `>` and `>>`?
- 4 Why do we put `sudo` in front of `apt install`?
- 5 Which command would you use to watch a log file as new lines are added?
- 6 What do the letters `r`, `w`, and `x` mean in permissions?

The Software Development Life Cycle (SDLC)

Every app you use — WhatsApp, Swiggy, your bank app — went through the same stages before it reached your phone.

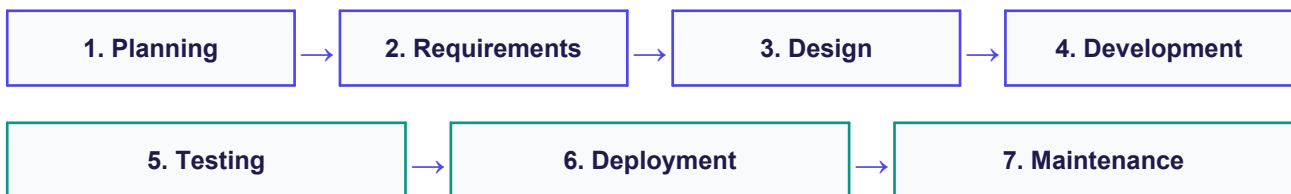
What is the SDLC?

The **Software Development Life Cycle (SDLC)** is the step-by-step process a team follows to plan, build, test, release, and look after software. It makes sure the software is useful, works correctly, and is delivered on time.

Think of it like this

Building software is like **building a house**: you decide what the family needs, draw a plan, build it, inspect it, hand over the keys, and then repair things over the years. Skipping a step causes problems later.

The seven phases



Phase	What happens	House example	Output
1. Planning	Decide the goal, budget, timeline, and team. Is the project worth doing?	Decide to build a 2-bedroom house with a budget	Project plan
2. Requirements	Talk to users and write down exactly what the software must do	List rooms, kitchen size, parking	Requirements document (SRS)
3. Design	Decide how it will work: screens, database, architecture	Architect draws the blueprint	Design documents, mock-ups
4. Development	Programmers write the code	Builders lay bricks	Working code
5. Testing	Testers check everything and report bugs	Inspector checks wiring and plumbing	Test reports, bug fixes
6. Deployment	Software is released to real users	Family moves in	Live application

Phase	What happens	House example	Output
7. Maintenance	Fix bugs, update, and add small features over time	Repairs and painting over the years	Updates, patches

People in a software team

Role	What they do
Client / Stakeholder	The person or business who wants the software
Business Analyst	Understands the client's needs and writes requirements
Project Manager	Plans the work, timeline, and budget
Architect / Designer	Designs the structure and screens
Developer	Writes the code
Tester (QA)	Checks quality and finds bugs
Operations (Ops)	Installs the software on servers and keeps it running

Important words

- **Bug** — a mistake in the software that makes it behave wrongly.
- **Release** — a new version of the software given to users.
- **Deployment** — the act of putting software on a server so users can use it.
- **Environment** — a copy of the software for a purpose: **Development** (for building), **Testing / Staging** (for checking), and **Production** (real users).

SDLC models

The seven phases are always there, but teams can arrange them in different ways. These arrangements are called **SDLC models**. The two most famous are **Waterfall** and **Agile**. Others include the V-Model, Spiral, and Iterative models.

In short

SDLC = the process to plan, build, test, release, and maintain software.

Seven phases: Planning → Requirements → Design → Development → Testing → Deployment → Maintenance.

Waterfall and Agile are two different ways of running these phases.

Check yourself

- 1 What does SDLC stand for?
- 2 Name the seven phases in order.
- 3 In which phase are bugs mainly found?
- 4 What is the difference between a testing environment and production?

The Waterfall model

The oldest and simplest way to build software: one phase after another, in a straight line.

How Waterfall works

In the **Waterfall model**, each phase must be **fully finished** before the next one starts — just like water flowing down a waterfall, it never goes back up. The model was described by Winston Royce in 1970.



Think of it like this

Waterfall is like **printing wedding invitations**. You finalise every detail first, then print 500 cards. If you find a spelling mistake after printing, you must reprint everything — expensive and slow.

Advantages

- **Simple and easy to understand** — clear steps, clear order.
- **Clear documents** at every stage.
- **Easy to manage** — each phase has a fixed deadline and deliverable.
- **Good for fixed projects** where requirements will not change, such as government or construction-related systems.

Disadvantages

- **The customer sees the product only at the end**, often after many months.
- **Changes are very hard** and expensive once a phase is finished.
- **Testing happens late**, so bugs are found late and cost more to fix.
- **High risk** — if the requirements were wrong, the whole project can fail.
- **Long delivery time** — 6 to 18 months before users get anything.

When to use Waterfall

Good fit	Poor fit
Requirements are clear and will not change	Requirements are unclear or keep changing
Small, short projects	Large, long, complex products
Strict rules or contracts (e.g. government, medical devices)	Fast-moving markets such as mobile apps or e-commerce

In short

Waterfall = one phase at a time, no going back.

Simple and well documented, but slow and hard to change.

The customer sees working software only at the very end.

Check yourself

- 1 Why is it called "Waterfall"?
- 2 Give two advantages and two disadvantages of Waterfall.
- 3 Why is late testing a problem?

CHAPTER 5

The Agile model

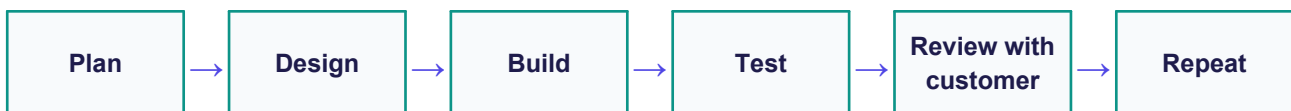
Build a little, show it, learn, and improve — again and again.

Why Agile was created

By the late 1990s, many Waterfall projects were late, over budget, or delivered something customers no longer wanted. In **2001**, seventeen software experts met and wrote the **Agile Manifesto**, a new way of working.

How Agile works

In **Agile**, the work is split into small pieces. Every 1 to 4 weeks the team builds a small, working part of the software and shows it to the customer. Feedback is used to plan the next piece.



Think of it like this

Agile is like **cooking for guests and tasting as you go**. You add a little salt, taste, adjust, and taste again — instead of cooking the whole meal and finding out at the table that it is too spicy.

The four Agile values

We value more...	...than
Individuals and interactions	processes and tools
Working software	comprehensive documentation
Customer collaboration	contract negotiation
Responding to change	following a plan

The items on the right still matter, but the items on the left matter more.

Scrum: the most popular Agile method

Three roles

Role	Responsibility
Product Owner	Decides what to build and in what order. Represents the customer.
Scrum Master	Helps the team follow Scrum and removes problems that block them.

Role	Responsibility
Developers	The team that designs, builds, and tests the product (usually 3–9 people).

Key words

Word	Meaning
Product Backlog	The full to-do list of features, most important at the top
User Story	A feature written from the user's view: "As a customer , I want to track my order so that I know when it arrives. "
Sprint	A fixed time box, usually 2 weeks, in which the team completes a set of user stories
Sprint Backlog	The stories chosen for the current sprint
Increment	The working piece of software finished at the end of a sprint

Scrum meetings (events)

Event	When	Purpose
Sprint Planning	Start of each sprint	Choose which stories to build in this sprint
Daily Stand-up	Every day, 15 minutes	Each person says: what I did yesterday, what I will do today, what is blocking me
Sprint Review	End of the sprint	Show the working software to the customer and get feedback
Sprint Retrospective	After the review	The team discusses how to work better next time



Kanban: another Agile method

Kanban uses a board with columns such as **To Do**, **In Progress**, and **Done**. Each task is a card that moves across the board. There are no fixed sprints; work flows continuously. Tools such as **Jira** and **Trello** are used for Scrum and Kanban boards.

To Do	In Progress	Done
-----	-----	-----
Login page	Payment page	Home page
Search		Product list

Waterfall vs Agile

Topic	Waterfall	Agile
Approach	One long straight line	Short repeating cycles (sprints)
Customer sees product	At the end	Every 1–4 weeks
Changes	Difficult and costly	Welcome at any time
Testing	After all coding is finished	Every sprint
Risk	High — problems found late	Lower — problems found early
Documentation	Heavy	Light, just enough
Best for	Fixed, clear requirements	Changing requirements, fast markets

The problem Agile did not solve

Agile made **developers** fast: new features were ready every two weeks. But the **operations** team, who put software on the servers, still worked the old way — with manual steps and releases only a few times a year.

Developers (Dev)	Operations (Ops)
Goal: deliver new features quickly	Goal: keep the live system stable
“The code is ready, release it!”	“Every change is a risk, slow down!”
“It works on my laptop.”	“It does not work on the server.”
Hands the code over and moves on	Gets the phone call at midnight when it breaks

The two teams sat on opposite sides of a **wall**. Code was ready in two weeks but reached customers months later, and failures turned into blame. This gap is the reason **DevOps** was born.

In short

Agile = small pieces, short sprints, constant customer feedback.

Scrum has 3 roles (Product Owner, Scrum Master, Developers) and 4 events (Planning, Stand-up, Review, Retrospective).

Agile made development fast, but releasing to servers was still slow and manual.

Check yourself

- 1 In which year was the Agile Manifesto written?
- 2 What is a sprint? How long is it usually?
- 3 Write a user story for a food-delivery app.
- 4 What happens in a daily stand-up?
- 5 Which problem was still left after teams adopted Agile?

Why DevOps?

DevOps removes the wall between the people who build software and the people who run it.

What is DevOps?

DevOps = Development + Operations. It is a way of working where developers and operations work as **one team**, share responsibility, and **automate** the journey of software from a developer's laptop to real users — so that changes can be released **quickly, often, and safely**.

Key idea

DevOps is a **culture and a way of working**, not a single tool or software. Tools help, but the main change is people working together.

Think of it like this

Think of a **restaurant**:

- **Waterfall**: plan a menu for the whole year, cook everything at once, serve at the end.
- **Agile**: cook small dishes and let the guest taste each one — but the waiters are a separate team who serve only once a month.
- **DevOps**: cooks and waiters work as one team. Each dish is cooked, checked, and served immediately, and guest feedback reaches the kitchen in minutes.

Waterfall → Agile → DevOps

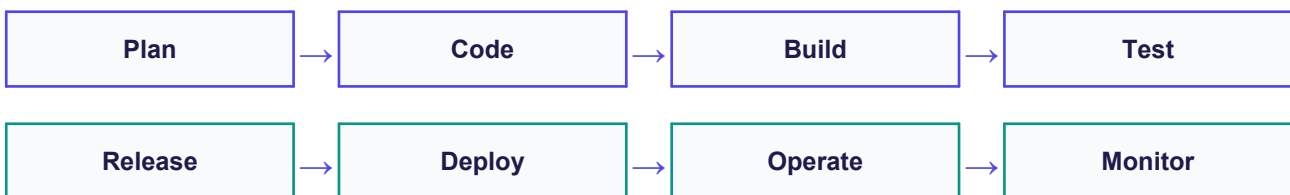
	Waterfall	Agile	DevOps
Release speed	Months or years	Weeks (but deployment still slow)	Days, hours, even minutes
Teams	Separate teams, hand-offs	Dev team works together	Dev + Ops + QA as one team
Testing	At the end	Every sprint	Automatic, on every change
Deployment	Manual, big and risky	Manual, slow	Automated, small and safe
Feedback	At the end	At sprint review	Continuous, from live users

The problems DevOps solves

Problem before DevOps	How DevOps solves it
Code waits weeks or months to be released	Automated pipelines release changes in minutes
Manual deployment steps cause mistakes	The same automated steps every time
"It works on my machine"	The same environment everywhere (for example with containers)
Bugs found late, by customers	Automatic tests run on every change
Dev and Ops blame each other	One team, shared goals: "you build it, you run it"
Failures take hours to notice	Monitoring and alerts warn the team immediately

The DevOps life cycle

DevOps is drawn as an **infinity loop** (∞) because the work never stops — every release teaches the team something for the next one.

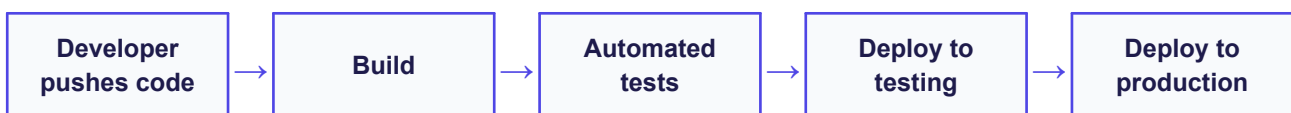


Stage	What happens	Example tools
Plan	Decide what to build	Jira, Trello
Code	Write code and save versions	Git, GitHub
Build	Turn code into a runnable package	GitHub Actions, Jenkins, Maven
Test	Run automatic tests	Selenium, JUnit
Release	Approve a version for users	GitHub Actions, Jenkins
Deploy	Put it on servers automatically	Docker, Kubernetes, AWS
Operate	Run and manage the servers	Terraform, Ansible, Linux
Monitor	Watch performance and errors	Prometheus, Grafana, CloudWatch

Key DevOps practices

- **Continuous Integration (CI)** — every time a developer saves code, it is automatically built and tested.
- **Continuous Delivery / Deployment (CD)** — code that passes the tests is automatically prepared for release or released.
- **Infrastructure as Code (IaC)** — servers and networks are created from code files, not by clicking (e.g. Terraform).
- **Monitoring and logging** — the team watches the live system and gets alerts when something goes wrong.
- **Collaboration** — Dev, Ops, and QA share tools, goals, and responsibility.

The automated chain of steps from code to users is called a **CI/CD pipeline**:



Think of it like this

A CI/CD pipeline is like a **car factory assembly line**: each station does one job automatically and checks quality before passing the car on. A faulty part is caught at the station, not by the customer.

Benefits of DevOps

Benefit	What it means for the business
Faster delivery	New features reach customers in days, not months
Better quality	Automatic tests catch bugs before customers see them
Less risk	Small changes are easy to check and easy to undo (rollback)
Faster recovery	Problems are noticed and fixed quickly
Lower cost	Less manual work and fewer failures
Happier teams	Less blame, less late-night firefighting
Happier customers	Fewer outages and quicker improvements

Who uses DevOps?

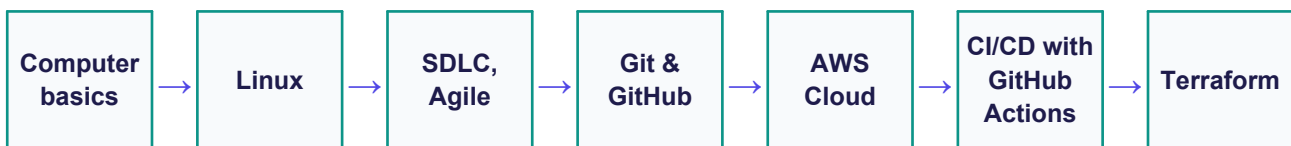
Companies such as **Amazon**, **Netflix**, **Google**, **Flipkart**, and most banks and startups use DevOps. Large companies release changes to their systems many times a day — something impossible with Waterfall.

What a DevOps engineer does

- Builds and maintains **CI/CD pipelines**.
- Manages **Linux servers** and the **cloud** (AWS, Azure, Google Cloud).
- Writes **Infrastructure as Code** (Terraform).
- Packages applications with **Docker** and runs them with **Kubernetes**.
- Sets up **monitoring and alerts**.
- Helps developers and operations work together smoothly.

Your learning path from here

The first three boxes are what you have learned in these notes. The rest come next.



In short

DevOps = Development + Operations working as one team, with automation.

It solves the slow, manual, risky release problem that remained after Agile.

Core practices: CI/CD, Infrastructure as Code, monitoring, collaboration.

Result: faster, safer, more frequent releases and happier customers.

Check yourself

- 1 What does DevOps stand for?
- 2 Is DevOps a tool? Explain.
- 3 Name the eight stages of the DevOps life cycle.
- 4 What is the difference between CI and CD?
- 5 Give three benefits of DevOps for a business.

REVISION

One-page revision sheet

Read this the night before an interview or test.

Topic	Remember this
Computer	Input → Process → Store → Output. CPU = brain, RAM = temporary, storage = permanent.
Sizes	8 bits = 1 byte. 1 KB = 1,024 bytes, 1 MB = 1,024 KB, 1 GB = 1,024 MB, 1 TB = 1,024 GB.
Operating system	Manages hardware and apps. Servers mostly run Linux; Ubuntu is a popular Linux.
Terminal	Type commands instead of clicking. <code>pwd</code> , <code>ls</code> , <code>cd</code> , <code>mkdir</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>cat</code> .
Admin and software	<code>sudo</code> = run as admin. <code>sudo apt install <name></code> installs software.
System checks	<code>df -h</code> disk, <code>free -h</code> memory, <code>top</code> processes, <code>ip a</code> network, <code>systemctl status</code> services.
SDLC	Planning → Requirements → Design → Development → Testing → Deployment → Maintenance.
Waterfall	One phase at a time, no going back. Simple, but slow and hard to change.
Agile	Short sprints, working software every 1–4 weeks, customer feedback, welcomes change.
Scrum	Roles: Product Owner, Scrum Master, Developers. Events: Planning, Stand-up, Review, Retro.
The gap	Agile made coding fast, but deployment stayed slow and manual. Dev and Ops were divided.
DevOps	Dev + Ops as one team, automating build, test, and deployment to release fast and safely.
CI/CD	CI = auto build and test every change. CD = auto release of tested changes.
Life cycle	Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (repeat).

ANSWERS

Answers to “Check yourself”

Short answers. Any answer that shows you understand the idea is correct.

Chapter 1 — Computer basics

- 1 It is lost. RAM is temporary memory.
- 2 Hardware: keyboard, monitor, CPU. Software: Windows, Chrome, WhatsApp.
- 3 Linux.
- 4 It converts a domain name (like google.com) into an IP address.

Chapter 2 — Linux and Ubuntu

- 1 `pwd`.
- 2 `mkdir website` then `cd website`.
- 3 `>` replaces the file's content; `>>` adds to the end of the file.
- 4 Installing software changes the system, so it needs administrator permission.
- 5 `tail -f filename`.
- 6 `r` = read, `w` = write (change), `x` = execute (run).

Chapter 3 — SDLC

- 1 Software Development Life Cycle.
- 2 Planning, Requirements, Design, Development, Testing, Deployment, Maintenance.
- 3 Testing.
- 4 Testing is a safe copy used to check the software; production is the live system used by real customers.

Chapter 4 — Waterfall

- 1 Because work flows down from one phase to the next and never goes back up, like water.
- 2 Advantages: simple, well documented, easy to manage. Disadvantages: hard to change, customer sees the product late, high risk.
- 3 Bugs are found after all the code is written, so they are harder and more expensive to fix.

Chapter 5 — Agile

- 1 2001.
- 2 A fixed time box in which the team builds a small working piece of software; usually 2 weeks.
- 3 Example: “As a hungry customer, I want to see the delivery time so that I can plan my meal.”
- 4 Each team member shares what they did yesterday, what they will do today, and anything blocking them.
- 5 Releasing software to servers was still slow and manual, and Dev and Ops worked as separate teams.

Chapter 6 — Why DevOps

- 1 Development and Operations.
- 2 No. DevOps is a culture and way of working. Tools such as GitHub Actions or Terraform support it.
- 3 Plan, Code, Build, Test, Release, Deploy, Operate, Monitor.
- 4 CI automatically builds and tests every code change; CD automatically prepares or releases changes that passed the tests.
- 5 Faster delivery, better quality, lower risk, quicker recovery, lower cost.

Glossary

Word	Meaning
Agile	Building software in short cycles with frequent feedback
apt	Ubuntu's tool for installing software
Bug	A mistake in software
CI/CD	Continuous Integration and Continuous Delivery/Deployment
Cloud	Renting computers and storage over the internet
CPU	The processor; the computer's brain
Deployment	Putting software on a server for users
DevOps	Development and Operations working as one team with automation
Directory	Another word for folder
Distribution	A packaged version of Linux, such as Ubuntu
Kanban	An Agile method using a board of To Do, In Progress, and Done
Linux	A free operating system used by most servers

Word	Meaning
Pipeline	An automated chain of steps from code to users
Process	A program that is currently running
Production	The live system used by real customers
RAM	Temporary working memory
Root	The Linux super user; also the top folder /
Scrum	The most popular Agile method, using sprints
SDLC	Software Development Life Cycle
Server	A computer that provides services to other computers
Service	A program that runs in the background all the time
Sprint	A short, fixed work period in Scrum, usually 2 weeks
sudo	Run a command with administrator rights
Terminal	A window for typing commands
Ubuntu	A popular, beginner-friendly Linux distribution
User story	A feature described from the user's point of view
Waterfall	Building software one phase at a time, in a straight line